AL MAAREF UNIVERSITY

Mastering Life with Wisdom & Knowledge

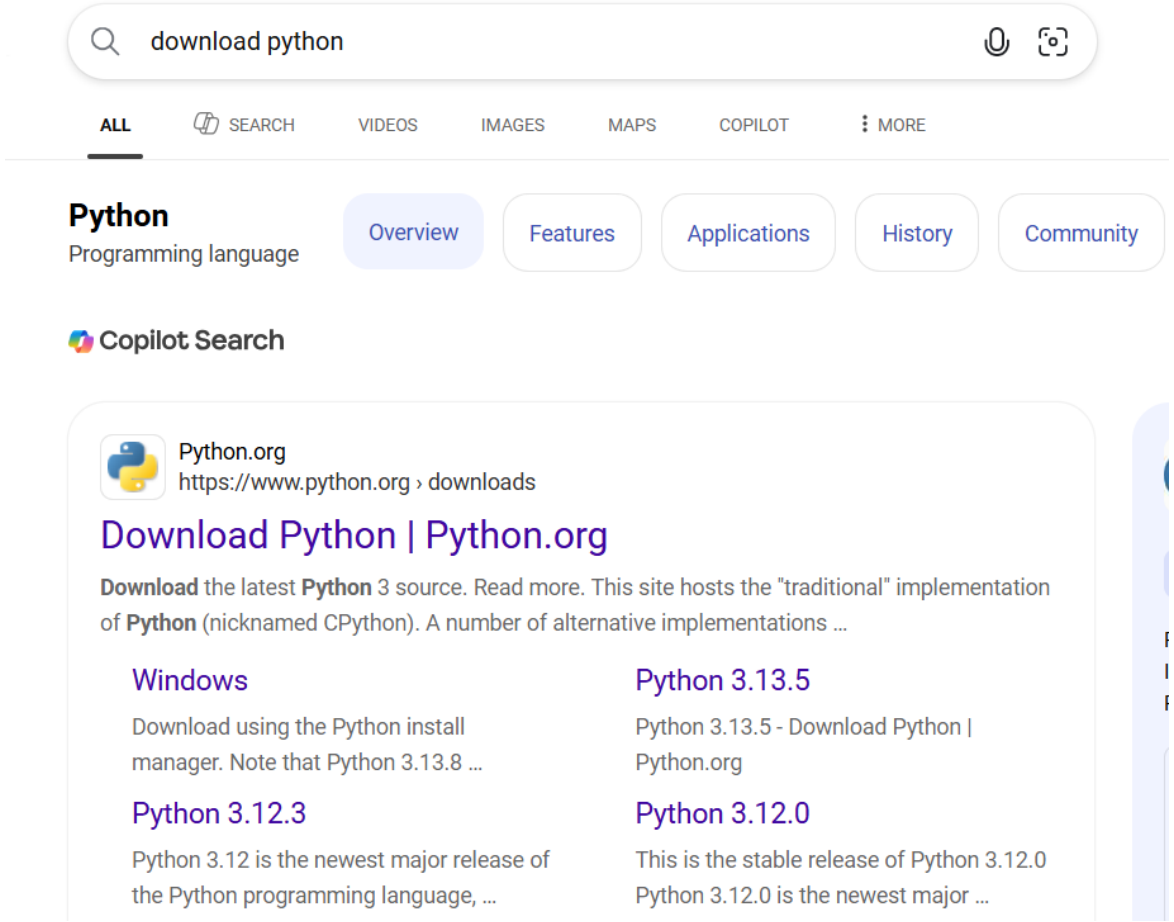
بِسْمِ ٱللَّهِ ٱلرَّحْمَٰنِ ٱلرَّحِيمِ

Welcome to HackathonTech 2026!

Let’s start our programming journey together in python 🚀

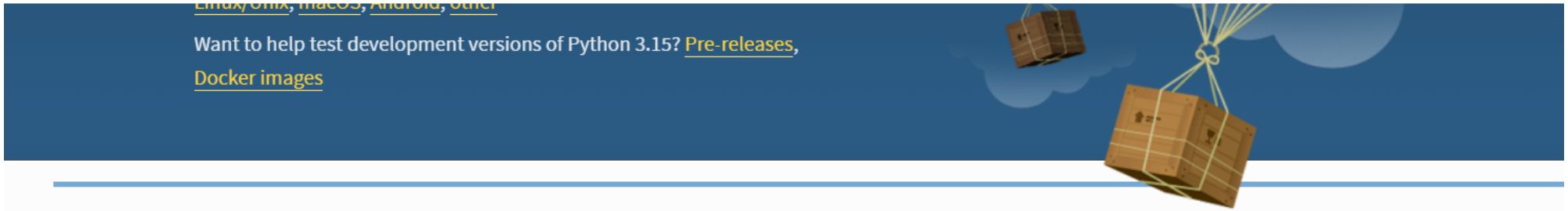
How to Setup Python step by step?

Step 1:



How to Setup Python step by step?

Step 2:



Active Python releases

For more information visit the Python Developer's Guide.

Python version	Maintenance status		First released	End of support	Release schedule
3.15	pre-release	Download	2026-10-07 (planned)	2031-10	PEP 790
3.14	bugfix	Download	2025-10-07	2030-10	PEP 745
3.13	bugfix	Download	2024-10-07	2029-10	PEP 719
3.12	security	Download	2023-10-02	2028-10	PEP 693
3.11	security	Download	2022-10-24	2027-10	PEP 664
3.10	security	Download	2021-10-04	2026-10	PEP 619
3.9	end of life, last release was 3.9.25	Download	2020-10-05	2025-10-31	PEP 596

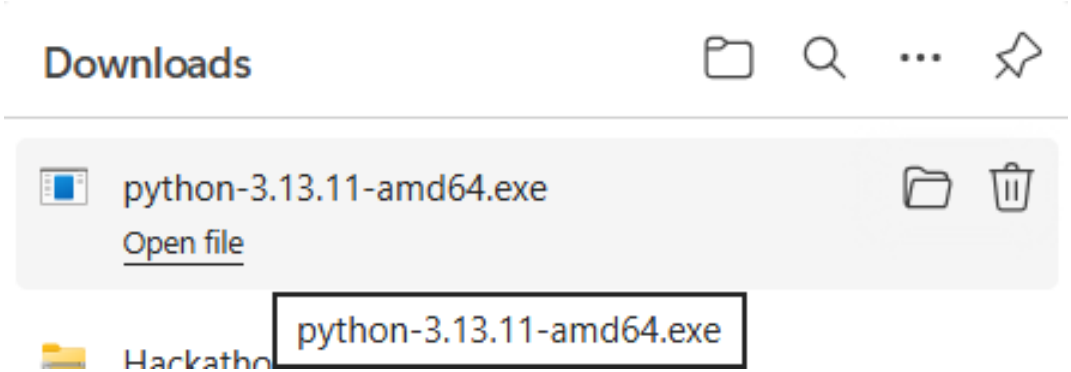
How to Setup Python step by step?

Step 3: Go down to the table and choose based on your laptop Operating System Version:

Version	Operating System	Description	MD5 Sum	File Size	Sigstore	SBOM	GPG
Gzipped source tarball	Source release		b0bea9e599a10ab3593ab60ace27b25d	28.0 MB	.sigstore	SPDX	SIG
XZ compressed source tarball	Source release		4c3517dd8b1fd76377dcb3e5e8f71ad6	21.7 MB	.sigstore	SPDX	SIG
macOS installer	macOS	for macOS 10.13 and later	df0282591c65374c3230363e59ee8ca6	67.0 MB	.sigstore		SIG
Windows installer (64-bit)	Windows	Recommended	379a32861f9f0a327f9fa9c163455574	27.4 MB	.sigstore	SPDX	SIG
Windows installer (32-bit)	Windows		41c1b631ed8db2d12f1c484e4a040984	26.2 MB	.sigstore	SPDX	SIG
Windows installer (ARM64)	Windows	Experimental	6058e2f1312b89b4355d56611207a77e	26.8 MB	.sigstore	SPDX	SIG
Windows embeddable package (64-bit)	Windows		31d4552395818632076260d39c5ce6cf	10.4 MB	.sigstore	SPDX	SIG
Windows embeddable package (32-bit)	Windows		c289058e529d33d6afb737cbb419c399	9.2 MB	.sigstore	SPDX	SIG
Windows embeddable package (ARM64)	Windows		69ed71db778f2d837f07765f1475583b	9.9 MB	.sigstore	SPDX	SIG
Windows release manifest	Windows	Install with 'py install 3.13'	357284b0d3cd5f5f208278ea151e83af	14.6 KB	.sigstore		

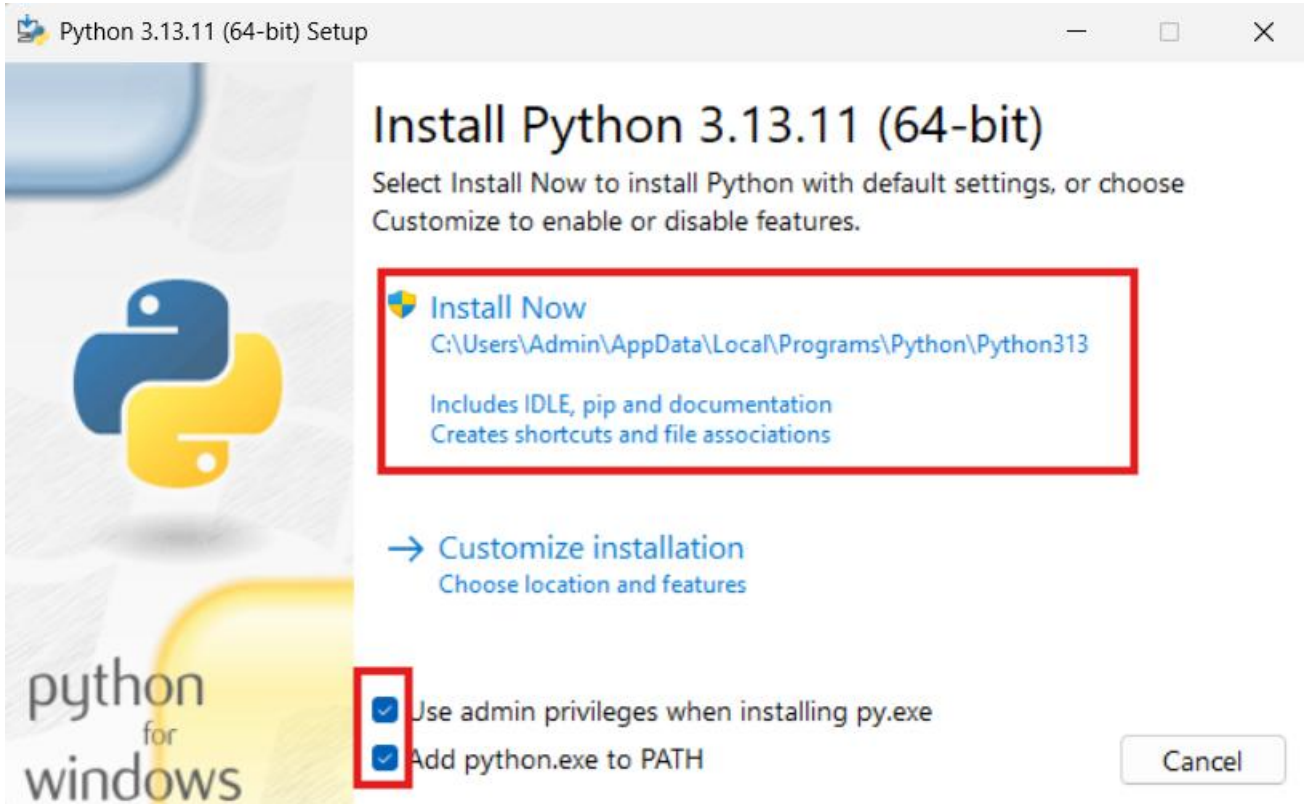
How to Setup Python step by step?

Step 4: Press Open File



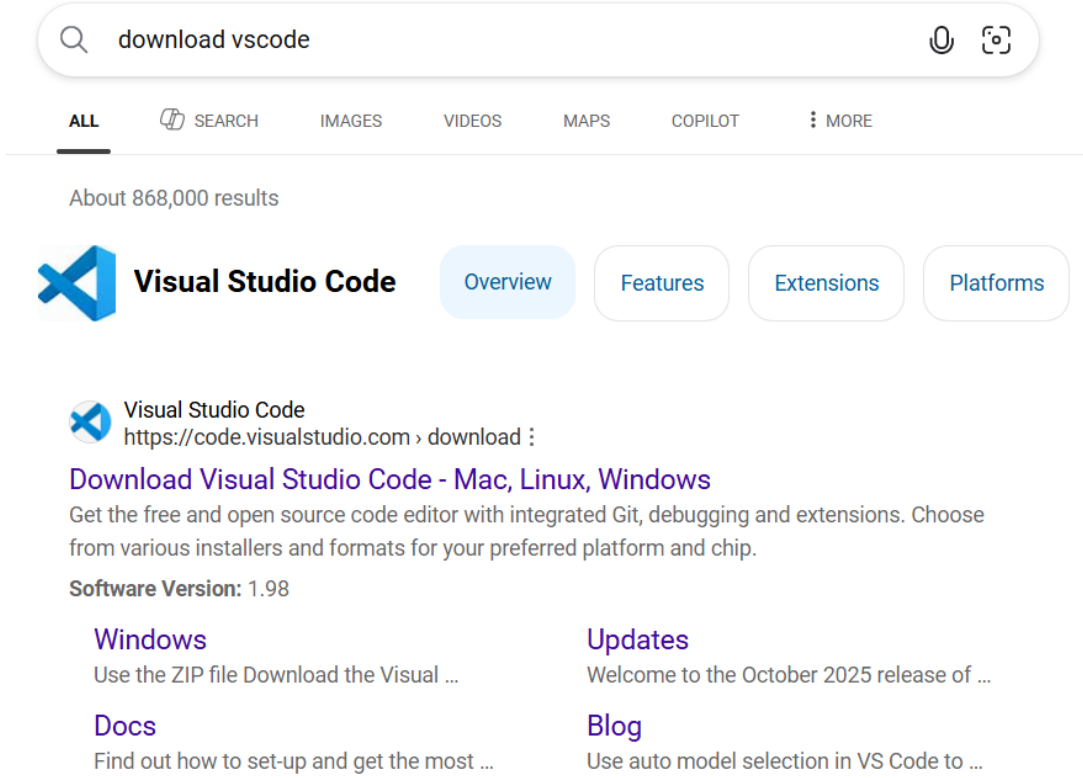
How to Setup Python step by step?

Step 5: Choose the checkbox buttons below then install now



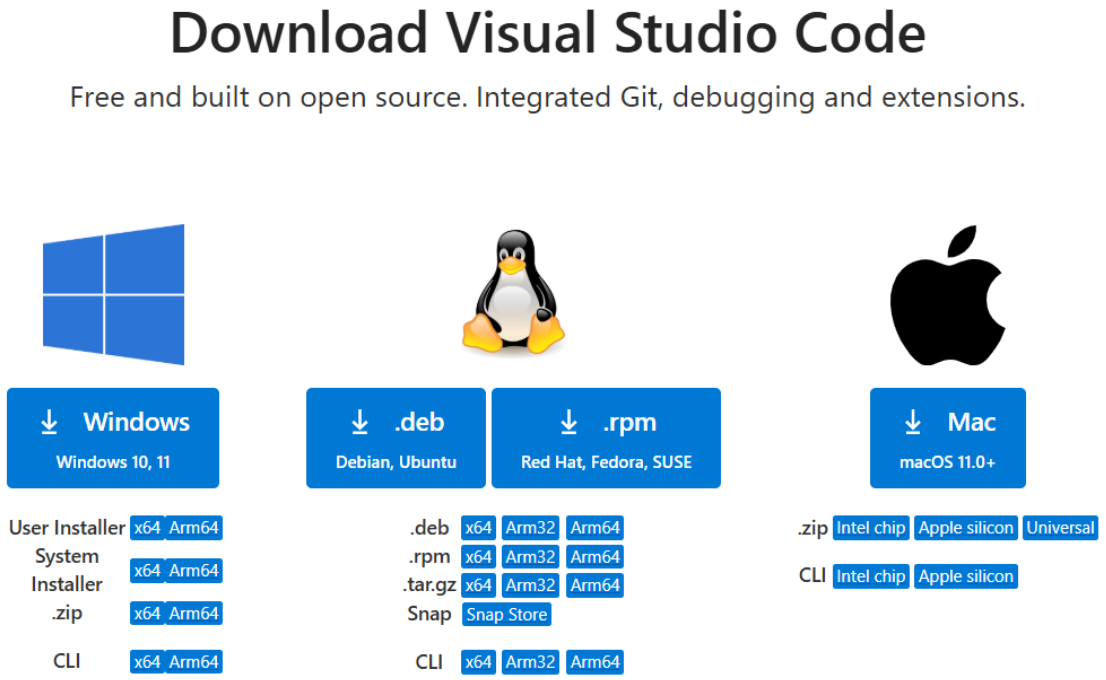
How to Setup Python step by step?

Step 6: Download visual studio code



How to Setup Python step by step?

Step 7: choose based on your laptop Operating System Version:



How to Setup Python step by step?

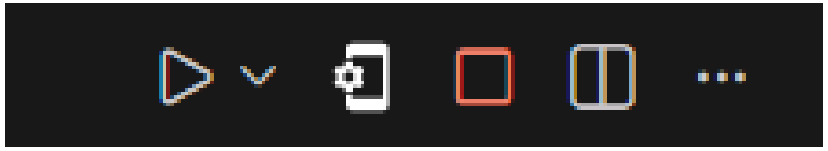
Setp 8: after opening vscode, install the following extensions

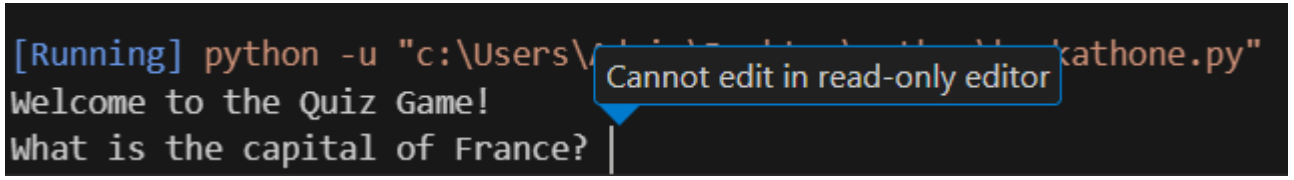
How to Setup Python step by step?

Step 9: also, this extension

How to Setup Python step by step?

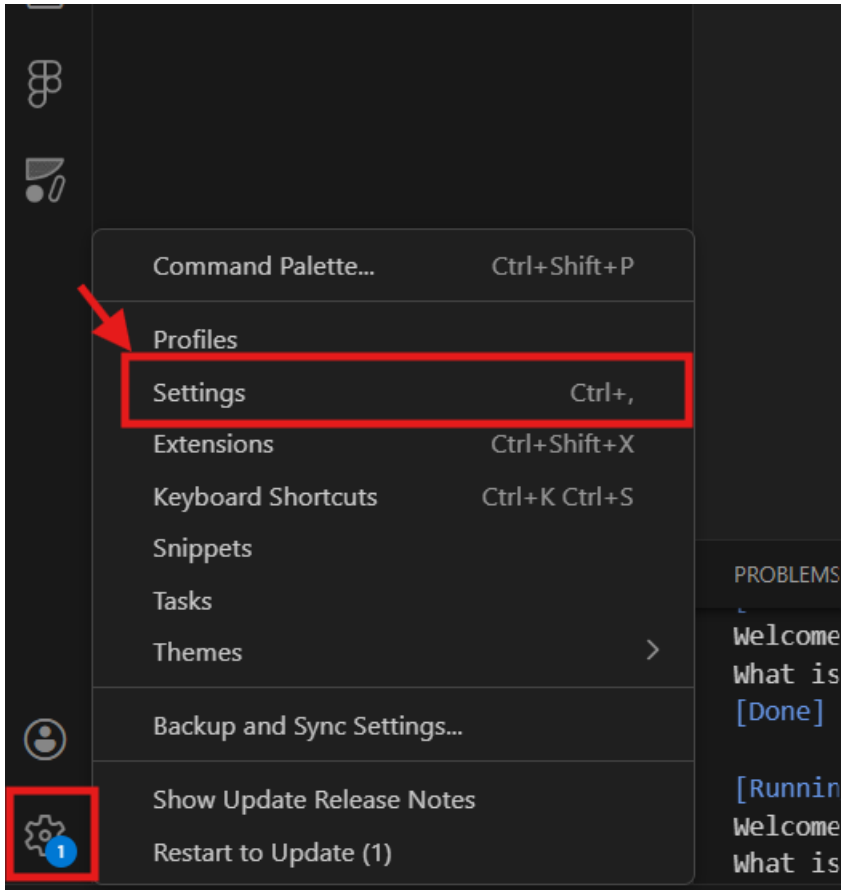
Step 10: you will get a run icon, but as you see we can’t write below, so we will solve this after that





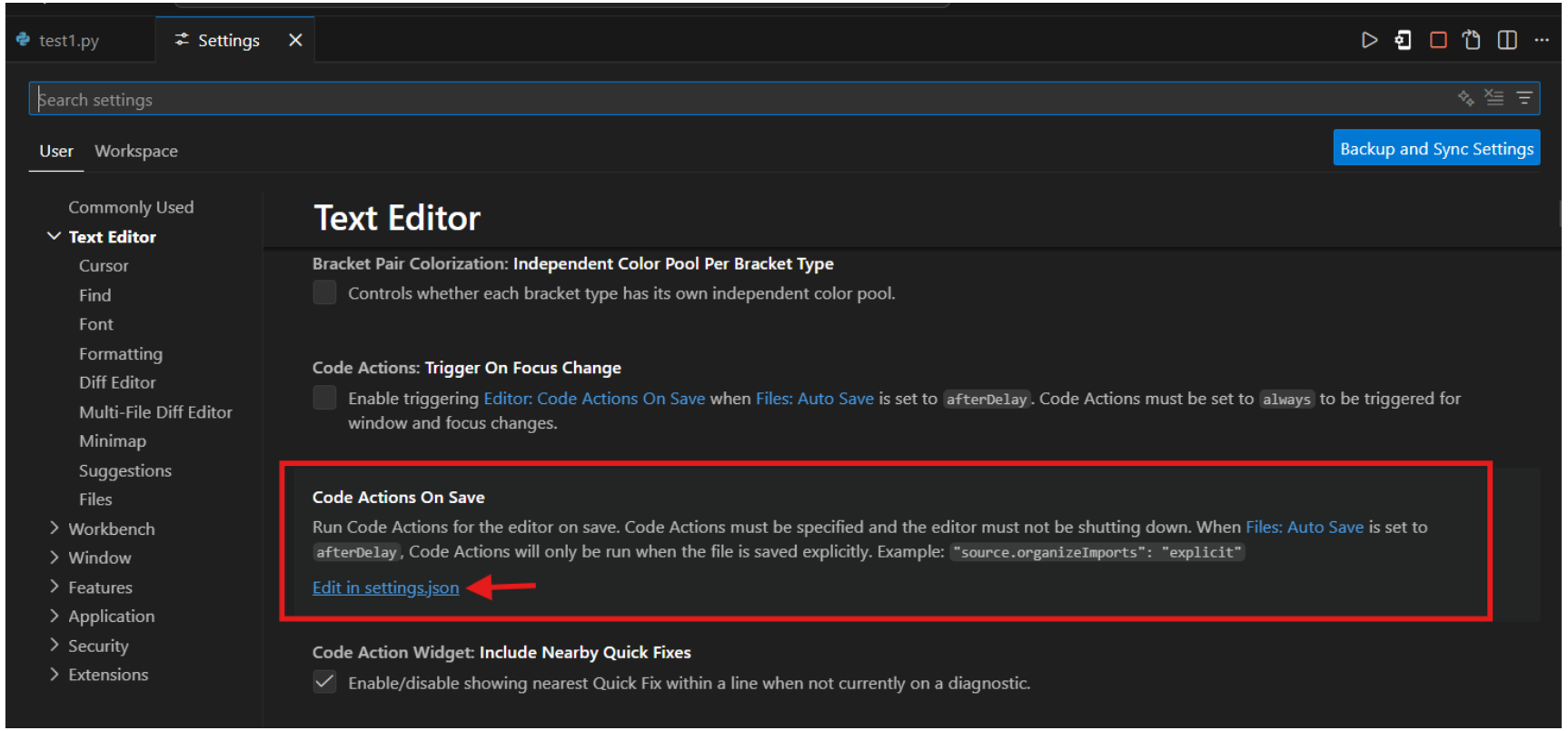
How to Setup Python step by step?

Step 11: go to settings



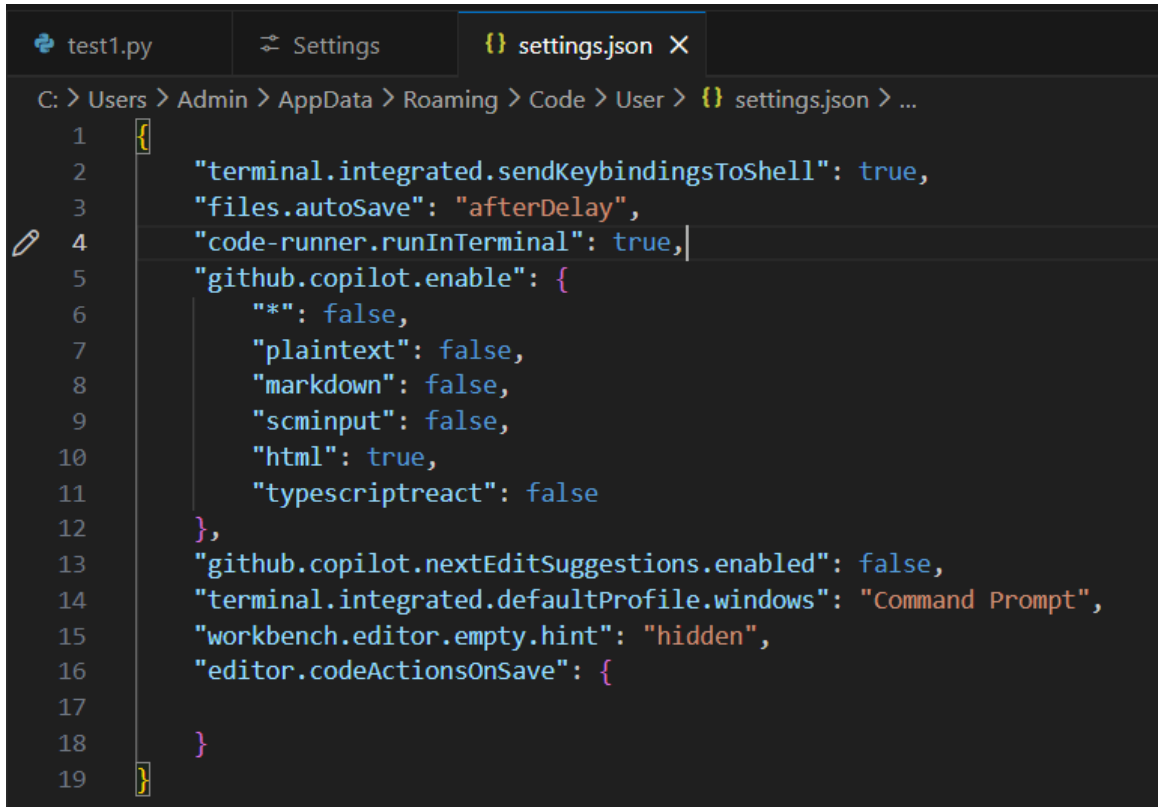
How to Setup Python step by step?

Step 12: scroll down until you see this:



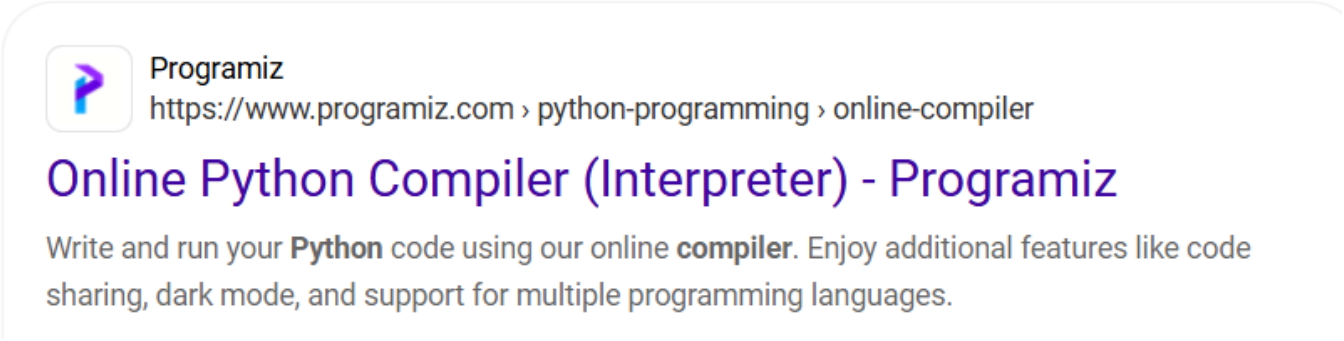
How to Setup Python step by step?

Step 13: Add the following line: "code-runner.runInTerminal": true,



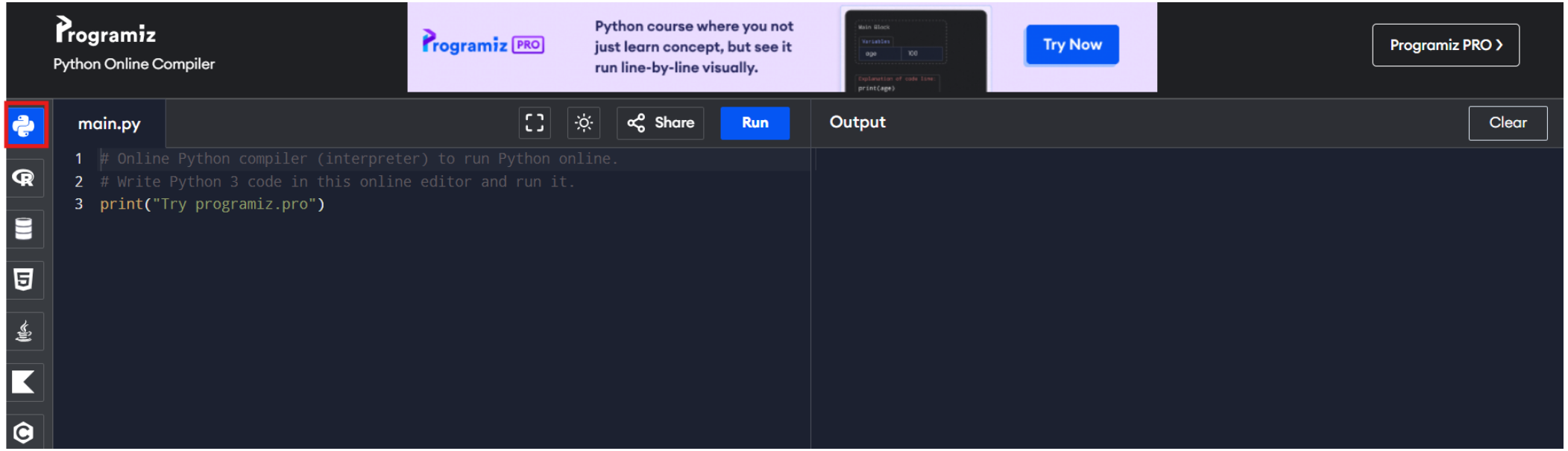
How to Setup Python step by step?

If you want to work online, use Programiz



How to Setup Python step by step?

Pay attention to chosen language



What is Python?

First, **what is programming?**

→ Giving instructions to a computer so it can do tasks.

Python is a programming language

Very popular and easy to learn

Used for:

- Websites (YouTube, Instagram…)
- Games
- Robots
- Artificial Intelligence

Fun Fact: The name “Python” came from a *comedy show*, not the snake!

Python is clean — **no semicolons**

Your First Code

❗ Indentation is VERY important in Python (spaces matter!)

Let’s write our first program: print Hello World

Mini challenge: Print any sentence you want

Mini Challenge

hackathone.py

```
1    print("Hello World!")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
Microsoft Windows [Version 10.0.26100.7171]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Admin\Desktop\python>python hackathone.py
Hello World!
```

Comments

First, **What are comments?**

→ Notes for programmers; Python ignores them.

Why use comments?

→ To explain code

How to write them?

This is a single comment
“”” This is a multiple comment “””

```
hackathone.py
1  # print hello world
2  print("Hello World!")
3
4  """
5      Python is very important language
6      used in AI
7      ...
8  """
```

Variables

Variables = **boxes that store information**

Types of variables: **str, int, float, bool**

Declare & assign:

```
4    name = "Fatima"
5    age = 15
```

Check type:

```
7    print(type(name))
```

Print with variables

```
9    print("Hello", name)
```

Input & Output

Input: Take data from the user

Output: Show data to the user

Mini Game:
Enter 3 colors → print them together

```
1   color = input("Enter your favorite color: ")
2   print("Your color is", color)
3
```

Mini Game

```
 9
10   color1 = input("Enter your first top color: ")
11   color2 = input("Enter your second top color: ")
12   color3 = input("Enter your third top color: ")
13
14   print("Your first three top colors are: ", color1, " ", color2, " ", color3)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
C:\Users\Admin\Desktop\python>python hackathone.py
Enter your first top color: blue
Enter your second top color: black
Enter your third top color: white
Your first three top colors are:  blue   black   white
```

Built-in String Functions

Case conversion: .upper(), .lower(), .capitalize(), .title()

```
text = "Hello World"
print(text.upper())  # HELLO WORLD
print(text.lower())  # hello world
print(text.capitalize()) # Hello world
print(text.title()) # Hello World
```

Trimming spaces: .strip()

```
name = "   Mahdi   "
print(name.strip())  # "Mahdi"
```

Searching: .find()

```
word = "programming"
print(word.find("gram"))  # 3 (found at index 3)
```

Replacing: .replace()

```
msg = "I like Java"
print(msg.replace("Java", "Python"))  # I like Python
```

Checking: .isdigit(), .isalpha(), etc

```
print("123".isdigit())  # True
print("Hello".isalpha())  # True
print("Hi123".isalpha())  # False
```

Mini Game

1. Ask the user for their name in messy format and fix it using string functions
2. Ask the user for any word and print how many characters it has
3. Ask the user to enter a sentence, then search if the word "python" exists inside it
4. Ask the user for a password. Check if it contains only numbers or only letters


```python
name = input("Enter your name in messy format: ")
print("Your name is: ", name.strip().title())

word = input("Enter any word to give you the length: ")
print(len(word))

sentence = input("Enter any sentence: ")
# print the index where the word is found
print(sentence.lower().find("python"))

password = input("enter your password: ")
print("Is it only numbers? ", password.isdigit())
print("Is it only letters? ", password.isalpha())
```

```
Your name is:  Fatima Alzahraa
Enter any word to give you the length: testhello
9
Enter any sentence: i am studying python
14
enter your password: test
Is it only numbers?  False
Is it only letters?  True
```

Operators

Math Operators:

- + addition
- - subtraction
- * multiplication
- / division
- // floor division
- % remainder
- ** power

Mini Games:

Ask user for 2 numbers → print:

- Sum
- Product
- Average

Mini Game

```python
num1 = float(input("Enter first number: "))
num2 = float(input("Enter second number: "))

sum_result = num1 + num2
product_result = num1 * num2
average_result = (num1 + num2) / 2
floor_average_result = (num1 + num2) // 2

print("Sum:", sum_result)
print("Product:", product_result)
print("Average:", average_result)
print("Floor Average:", floor_average_result)
```

```
Enter first number: 4
Enter second number: 5
Sum: 9.0
Product: 20.0
Average: 4.5
Floor Average: 4.0
```

And Now with Our Mini Project

Let’s do an Age Calculator! 🤩

1. Enter your birth year
2. Get current year using datetime
3. Output age

datetime is a built-in Python library used for working with dates and times.

Hint: import datetime first.

Mini Game

```
77   import datetime
78
79   birth_year = int(input("Enter your birth year: "))
80   current_year = datetime.datetime.now().year
81
82   age = current_year - birth_year
83   print("Your age is:", age)
84
```

```
Enter your birth year: 2005
Your age is: 20
```

Some Questions!

1. What does print() do?
2. How do you write a comment?
3. What is the difference between int and str?

Bonus: Calculate based on this function: $x^2 + 4x + 8$.

Bonus

```python
x = float(input("Enter a value for x: "))

result = x**2 + 4*x + 8

print("The result is:", result)
```

```
C:\Users\Admin\Desktop\python>python hackathone.py
Enter a value for x: 4
The result is: 40.0

C:\Users\Admin\Desktop\python>python hackathone.py
Enter a value for x: 2
The result is: 20.0
```

Are you ready for a new idea?

Logic with IF

First, **What are conditions?**

→ Conditions let Python **make decisions**

→ They check *if something is true* → then perform actions

What is the syntax?

→ If, elif, else

```
93   x = 10
94
95   if x > 10:
96       print("x is greater than 10")
97   elif x == 10:
98       print("x is equal to 10")
99   else:
100      print("x is less than 10")
101
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
C:\Users\Admin\Desktop\python>python hackathone.py
x is equal to 10
```

Operators

Comparison Operators

- == equal
- != not equal
- > greater than
- < less than
- >= greater or equal
- <= less or equal

```
age = 15

print(age == 15)    # True
print(age != 18)    # True
print(age > 10)     # True
print(age < 20)     # True
print(age >= 15)    # True
print(age <= 14)    # False
```

Logical Operators

- and
- or
- not

```
x = 8

print(x > 5 and x < 10)    # True
print(x > 10 or x == 8)    # True
print(not (x == 8))        # False
```

Application: Temperature Checker

1. **Ask the user** to enter the temperature.
2. **Use if / elif / else** to check the temperature range.
3. **Print a message** based on the temperature:
    1. Above 30 → "Very hot 🔥"
    2. Between 20–30 → "Warm 🌤️"
    3. Below 10 → "Very cold ❄️"
4. **Run the program** and test with different numbers.

Challenge: Add a "**rainy** ☔" case for temperature between **10–20**

Temperature Checker

```python
temp = int(input("Enter the temperature: "))

if temp > 30:
    print("It's very hot 🔥")
elif temp > 20:
    print("The weather is warm 🌤️")
elif temp >= 10:
    print("It's a bit cold and rainy 🌧️")
else:
    print("It's very cold ❄️")
```

```
C:\Users\Admin\Desktop\python>python hackathone.py
Enter the temperature: 30
The weather is warm 🌤️

C:\Users\Admin\Desktop\python>python hackathone.py
Enter the temperature: 35
It's very hot 🔥

C:\Users\Admin\Desktop\python>python hackathone.py
Enter the temperature: 20
It's a bit cold and rainy 🌧️

C:\Users\Admin\Desktop\python>python hackathone.py
Enter the temperature: 5
It's very cold ❄️
```

Switch

First, **What is switch?**

→ used to compare a value against multiple possible cases

→ starting from python 3.10 we call it match!

Mini Game: enter number of day and return the day

What is the syntax?

```
11  match x:
12      case 1:
13          print("One")
14      case 2:
15          print("Two")
16      case _:
17          print("Other")
18
```

Mini Game

```python
day_num = int(input("Enter a number (1-7): "))

match day_num:
    case 1:
        print("Monday")
    case 2:
        print("Tuesday")
    case 3:
        print("Wednesday")
    case 4:
        print("Thursday")
    case 5:
        print("Friday")
    case 6:
        print("Saturday")
    case 7:
        print("Sunday")
    case _:
        print("Invalid number! Please enter 1-7.")
```

```
C:\Users\Admin\Desktop\python>python hackathone.py
Enter a number (1-7): 5
Friday

C:\Users\Admin\Desktop\python>python hackathone.py
Enter a number (1-7): 0
Invalid number! Please enter 1-7.

C:\Users\Admin\Desktop\python>python hackathone.py
Enter a number (1-7): 7
Sunday
```

What are Loops?

For Loop

First, **What is for loop?**

→ used to repeat a block of code **a specific number of times**

→ It goes through a sequence (like numbers, lists, or strings) **one item at a time**

What is the syntax?

```
for i in range(start, stop, step):
    # code
```

→ range() generates a sequence of numbers

Mini Game:

- print numbers from 1 to 10
- Print even numbers from 0 to 10

Mini Games

```python
for i in range(1, 11):
    print(i)
```

```
C:\Users\Admin\Desktop\python>python hackathone.py
1
2
3
4
5
6
7
8
9
10
```

```python
for i in range(0, 11, 2):
    print(i)
```

```
C:\Users\Admin\Desktop\python>python hackathone.py
0
2
4
6
8
10
```

While Loop

First, **What is while loop?**

→ repeats a block of code **as long as a condition is true**

→ used when you *don't know how many times* you need to repeat something

Mini Game: Let's do a countdown application

What is the syntax?

```
22  while condition:
23      # code
```

→ Condition must eventually become false
→ Otherwise → infinite loop

Mini Challenge: Make a guessing loop, ask the user until they enter a specific number

Mini Game

```
count = 10

while count > 0:
    print(count)
    count -= 1

print("Go! ⏰")
```

Mini Challenge

```
secret_number = 7
guess = 0

while guess != secret_number:
    guess = int(input("Guess the number: "))

print("Congratulations! You guessed it! 🎉")
```

More Complex

```
secret_number = 7
guess = int(input("Guess the number: "))

while guess != secret_number:
    if (guess < secret_number):
        guess = int(input("Greater! Guess the number: "))
    else:
        guess = int(input("Smaller! Guess the number: "))

print("Congratulations! You guessed it! 🎉")
```

Lists

First, **What are lists?**

→ **collections of items** stored in one variable

→ They can hold **numbers, words, or mixed data**, and they are **ordered, changeable**, and **index-based**

Declaration?

```
25    fruits = ["apple", "banana", "mango"]
```

Indexing?

```
27    print(fruits[0])
28    print(fruits[-1])    # last element
```

Lists

Looping Over Lists

```
30    for item in fruits:
31        print(item)
```

Slicing

```
33    list[start : stop : step]
```

Built-in List Functions

append(value)

insert(index, value)

del list[index]

sort()

extend([value, value])

remove(value)

reverse()

count(value)

pop()

pop(index)

index(value)

if value in list

Application: Favorite fruits

Challenge:

1. Store your favorite fruits and print them
2. Reverse the list using reverse()

Application


```python
# Sample list
fruits = ["apple", "banana", "cherry"]

# 1. append(value) - Add at the end
fruits.append("orange")
print(f"After append: {fruits}")

# 2. insert(index, value) - Add at specific position
fruits.insert(1, "kiwi")
print(f"After insert: {fruits}")

# 3. extend([value, value]) - Add multiple items
fruits.extend(["mango", "grape"])
print(f"After extend: {fruits}")

# 4. remove(value) - Remove first occurrence
fruits.remove("banana")
print(f"After remove: {fruits}")

# 5. pop() - Remove last item
last_item = fruits.pop()
print(f"Popped item: {last_item}, List now: {fruits}")

# 6. pop(index) - Remove item at index
second_item = fruits.pop(1)
print(f"Popped index 1: {second_item}, List now: {fruits}")
```

```python
# 7. del list[0] - Delete by index
del fruits[0]
print(f"After del: {fruits}")

# 8. sort() - Sort ascending
numbers = [5, 2, 9, 1]
numbers.sort()
print(f"Sorted numbers: {numbers}")

# 9. reverse() - Reverse the list
numbers.reverse()
print(f"Reversed numbers: {numbers}")

# 10. count(value) - Count occurrences
count_apples = fruits.count("apple")
print(f"Apple count: {count_apples}")

# 11. index(value) - Find index of first occurrence
fruits = ["apple", "kiwi", "mango"]
index_kiwi = fruits.index("kiwi")
print(f"Index of kiwi: {index_kiwi}")

# 12. if value in list - Check existence
if "mango" in fruits:
    print("Mango is in the list!")
else:
    print("Mango not found!")
```

Mini Projects:

Mini Project 1: Shopping List Maker

- Ask user for **5 items or more**
- Store in a list
- Print the list
- **Bonus:** Add "Remove Item“ feature

Mini Project 2: Number Analyzer

- Ask for **5 numbers**
- Store in list
- Print:
  - Total
  - Largest number
  - Smallest number
  - Average

Shopping List Maker


```python
# Create an empty list
shopping_list = []

# Ask the user for 5 items or more
for i in range(5):
    item = input(f"Enter item {i+1}: ")
    shopping_list.append(item)

# Optionally ask if they want to add more items
while True:
    more = input("Do you want to add another item? (yes/no): ").lower()
    if more == "yes":
        item = input("Enter item: ")
        shopping_list.append(item)
    else:
        break
```

```python
# Print the shopping list
print("\nYour Shopping List:")
for item in shopping_list:
    print("-", item)

# Bonus: Remove item feature
remove_item = input("\nEnter an item to remove (or press Enter to skip): ")
if remove_item in shopping_list:
    shopping_list.remove(remove_item)
    print("\nUpdated Shopping List:")
    for item in shopping_list:
        print("-", item)
else:
    print("No item removed.")
```

Numbers Analyzer

```python
# Create an empty list
numbers = []

# Ask the user for 5 numbers
for i in range(5):
    num = float(input(f"Enter number {i+1}: "))
    numbers.append(num)

# Calculate total
total = sum(numbers)

# Find largest and smallest
largest = max(numbers)
smallest = min(numbers)

# Calculate average
average = total / len(numbers)

# Print results
print("\nNumbers entered:", numbers)
print("Total:", total)
print("Largest number:", largest)
print("Smallest number:", smallest)
print("Average:", average)
```

```
C:\Users\Admin\Desktop\python>python hackathone.py
Enter number 1: 5
Enter number 2: 6
Enter number 3: 7
Enter number 4: 20
Enter number 5: 34

Numbers entered: [5.0, 6.0, 7.0, 20.0, 34.0]
Total: 72.0
Largest number: 34.0
Smallest number: 5.0
Average: 14.4
```

List Comprehensions

First, **What are list comprehensions?**

→ **short and powerful way to create new lists** from existing lists

→ They let you **loop, filter, and transform data** in a single line of code

What is the syntax?

```
35    new_list = [expression for item in original_list if condition]
```

Mini Projects:

Simple Example:

```
37    numbers = [1, 2, 3, 4]
38    squares = [x*x for x in numbers]
```

Example with condition:

```
40    evens = [x for x in numbers if x % 2 == 0]
```

End of Day 2

Let’s do Rock–Paper–Scissors Game! 

Requirements:
1.Use **do while style loop** (while True: break)
2.Check each player’s choice
3.After **10 attempts**, show winner
4.Use:
1. conditions
2. loops
3. lists (optional to store results)

Hint: you can use random library to play with computer!

- random is a **built-in Python module** used to generate **random numbers or choices**.
- It is useful for **games, simulations, and testing**.

Rock–Paper–Scissors Game


```python
import random

# List of choices
choices = ["rock", "paper", "scissors"]

# Scores
player_score = 0
computer_score = 0

# Attempts
attempts = 0
max_attempts = 10

# Game loop (do-while style)
while True:
    # Check if max attempts reached
    if attempts >= max_attempts:
        break

    # Player input
    player_choice = input("Enter rock, paper, or scissors: ").lower()
    if player_choice not in choices:
        print("Invalid choice, try again!")
        continue

    # Computer choice
    computer_choice = random.choice(choices)
    print("Computer chose:", computer_choice)

    # Determine winner
    if player_choice == computer_choice:
        print("It's a tie!")
    elif (player_choice == "rock" and computer_choice == "scissors") or \
         (player_choice == "paper" and computer_choice == "rock") or \
         (player_choice == "scissors" and computer_choice == "paper"):
        print("You win this round!")
        player_score += 1
    else:
        print("Computer wins this round!")
        computer_score += 1

    attempts += 1
    print(f"Score -> You: {player_score}, Computer: {computer_score}")
    print(f"Attempt {attempts} of {max_attempts}\n")

# Final winner
print("Game Over!")
if player_score > computer_score:
    print("Congratulations! You won the game! 🎉")
elif computer_score > player_score:
    print("Computer won the game! 😳")
else:
    print("The game is a tie! 🤝")
```

Are you ready again for a new idea?

Dictionaries

First, **What is a dictionary?**

→ **collection of key–value pairs**
where each key has a specific value

First, **Why use dictionary?**

→ To store structured data

→ Fast access using keys

→ Easy to update and modify

What is the syntax?

```
42  my_dict = {
43      "key1": value1,
44      "key2": value2
45  }
```

```
47  student = {
48      "name": "Ali",
49      "age": 16,
50      "grade": "A"
51  }
```

Dictionaries

Accessing dictionary data

```
53    student["name"]
54    student.get("age")
```

→ **student[“name”]**

- Directly accesses the value.
- If the key does NOT exist → it gives an error

→ **student.get(“age”)**

- Safely accesses the value.
- If the key does NOT exist → it returns None (no error)

Basic Operations

→ Add: dict["key"] = value
→ Change: dict["key"] = new_value
→ Remove: del dict["key"]

Looping

```
56    for key, value in dict.items():
```

Dictionaries

Built-in dictionary functions:

→ **.keys()**
→ **.values()**
→ **.items()**
→ **.pop(key)**
→ **.update({…})**

Mini Application Options

→ **Student Info Program** (name, age, grade…)
→ **Vocabulary App** (word → meaning)

Student Info Program


```python
# Sample dictionary
student = {"name": "Ali", "age": 16, "grade": "A"}

# 1. .keys() - Get all keys
print("Keys:", student.keys())

# 2. .values() - Get all values
print("Values:", student.values())

# 3. .items() - Get key-value pairs
print("Items:", student.items())

# 4. .pop(key) - Remove a key and return its value
removed_value = student.pop("grade")
print("Removed value:", removed_value)
print("After pop:", student)

# 5. .update({...}) - Update or add key-value pairs
student.update({"age": 17, "city": "Beirut"})
print("After update:", student)
```

```
C:\Users\Admin\Desktop\python>python hackathone.py
Keys: dict_keys(['name', 'age', 'grade'])
Values: dict_values(['Ali', 16, 'A'])
Items: dict_items([('name', 'Ali'), ('age', 16), ('grade', 'A')])
Removed value: A
After pop: {'name': 'Ali', 'age': 16}
After update: {'name': 'Ali', 'age': 17, 'city': 'Beirut'}
```

Vocabulary App


```python
vocab = {}  # Empty dictionary to store word -> meaning

while True:
    print("\nOptions:")
    print("1. Add word")
    print("2. Look up word")
    print("3. Remove word")
    print("4. Show all words")
    print("5. Exit")

    choice = input("Choose an option (1-5): ")

    if choice == "1":
        word = input("Enter the word: ")
        meaning = input("Enter the meaning: ")
        vocab[word] = meaning
        print(f"{word} added to the vocabulary!")

    elif choice == "2":
        word = input("Enter the word to look up: ")
        if word in vocab:
            print(f"{word}: {vocab[word]}")
        else:
            print("Word not found!")

    elif choice == "3":
        word = input("Enter the word to remove: ")
        if word in vocab:
            vocab.pop(word)
            print(f"{word} removed!")
        else:
            print("Word not found!")

    elif choice == "4":
        print("\nVocabulary List:")
        for w, m in vocab.items():
            print(f"{w} → {m}")

    elif choice == "5":
        print("Exiting Vocabulary App. Bye!")
        break

    else:
        print("Invalid choice! Try again.")
```

Vocabulary App Output

```
Options:
1. Add word
2. Look up word
3. Remove word
4. Show all words
5. Exit
Choose an option (1-5): 1
Enter the word: hello
Enter the meaning: hi
hello added to the vocabulary!

Options:
1. Add word
2. Look up word
3. Remove word
4. Show all words
5. Exit
Choose an option (1-5): 1
Enter the word: black
Enter the meaning: not white
black added to the vocabulary!

Options:
1. Add word
2. Look up word
3. Remove word
4. Show all words
5. Exit
Choose an option (1-5): 4

Vocabulary List:
hello → hi
black → not white
```

```
Options:
1. Add word
2. Look up word
3. Remove word
4. Show all words
5. Exit
Choose an option (1-5): 2
Enter the word to look up: hello
hello: hi

Options:
1. Add word
2. Look up word
3. Remove word
4. Show all words
5. Exit
Choose an option (1-5): 3
Enter the word to remove: black
black removed!

Options:
1. Add word
2. Look up word
3. Remove word
4. Show all words
5. Exit
Choose an option (1-5): 4

Vocabulary List:
hello → hi
```

```
Options:
1. Add word
2. Look up word
3. Remove word
4. Show all words
5. Exit
Choose an option (1-5): 5
Exiting Vocabulary App. Bye!
```

Functions

First, **What are functions?**

→ named block of code that **performs a specific task**

→ allow you to **reuse code**, make programs **cleaner**, and **avoid repeating logic**

Mini Game: function that returns the square of a number

What is the syntax?

```
58    def greet(name):
59        print("Hello, " + name + "!")
```

→ def → declares the function
→ greet → function name
→ name → parameter, we can have default values

Mini Game


```python
def square(num):
    return num * num

# Test
result = square(5)
print(result)   # Output: 25
```

Return Values

First, **What is returning values?**

→ **sends a value back** from a function to the part of the program that called it

→ This allows you to **use the result** later in your code

Mini Game: Function that takes a grade (0–100) and returns A, B, C, D, F

What is the syntax?

```
61    def add(a, b):
62        return a + b
63
64    result = add(5, 3)
65    print(result)  # Output: 8
```

Mini Game


```python
def get_grade(score):
    if score >= 90:
        return "A"
    elif score >= 80:
        return "B"
    elif score >= 70:
        return "C"
    elif score >= 60:
        return "D"
    else:
        return "F"

# Test
print(get_grade(85))   # Output: B
```

Built-in Functions

Useful Built-in Functions

→ **len()**

→ **min()**

→ **max()**

→ **sum()**

→ **sorted()**

→ **random.randint()**

→ **type()**

Mini Game:

1. Let's generate random numbers
2. Find min/max of a list
3. Sort a list

Mini Game


```python
import random

# Random integer between 1 and 100
num = random.randint(1, 100)
print("Random number:", num)
```

```python
numbers = [5, 12, 3, 19, 7]

print("Minimum:", min(numbers))
print("Maximum:", max(numbers))
```

```python
numbers = [5, 12, 3, 19, 7]

# Ascending
numbers.sort()
print("Sorted list:", numbers)

# Descending
numbers.sort(reverse=True)
print("Descending:", numbers)
```

Mini Final Programs

1. Guess the Number

→ Use random.randint(1, 10)
→ Ask user to guess
→ Show number of attempts
→ Use loop + conditions

2. Calculator Program

→ Ask for two numbers
→ Ask for operation (+, -, *, /)
→ Create a function for each operation
→ Use switch (match-case) to choose the correct function

3. Quiz Game

→ Ask multiple questions
→ Check if answer is correct
→ Count score
→ Print final result

Guess the Number


```python
import random

secret = random.randint(1, 100)
attempts = 0

while True:
    guess = int(input("Guess a number between 1 and 100: "))
    attempts += 1

    if guess == secret:
        print("🎉 Correct! You guessed it in", attempts, "attempt(s).")
        break
    elif guess < secret:
        print("Too low! Try again.")
    else:
        print("Too high! Try again.")
```

Calculator Program


```python
def add(a, b):
    return a + b

def sub(a, b):
    return a - b

def mul(a, b):
    return a * b

def div(a, b):
    return a / b

a = float(input("Enter first number: "))
b = float(input("Enter second number: "))
op = input("Choose operation (+, -, *, /): ")

match op:
    case "+":
        print("Result =", add(a, b))
    case "-":
        print("Result =", sub(a, b))
    case "*":
        print("Result =", mul(a, b))
    case "/":
        print("Result =", div(a, b))
    case _:
        print("Invalid operation!")
```

Quiz Game


```python
score = 0

print("Welcome to the Quiz Game!")

questions = {
    "What is the capital of France? ": "paris",
    "How many legs does a spider have? ": "8",
    "What is 5 + 3? ": "8"
}

for q, correct_answer in questions.items():
    answer = input(q).lower()

    if answer == correct_answer:
        print("Correct!")
        score += 1
    else:
        print("Wrong!")

print("\nYour final score is:", score, "/", len(questions))
```

End of Python Basics Course!

But What is next🤩